FaaST.app

MANUAL GENERAL

# Los equipos de agentes de FaaST

Que hace cada equipo, como se instalan y como trabajan juntos: de lo que pide el cliente a un producto construido, probado, publicado y listo para vender.

**Marketplace FaaST** 10 de noviembre de 2026 faast.app

**Para quien es:** cualquier persona de FaaST que vaya a trabajar con los equipos de agentes: quien atiende clientes, quien dirige un area, quien vende, quien desarrolla y quien nunca abrio una herramienta tecnica. No necesitas saber programar.

| Equipo (plugin) | Version | Integrantes | Comandos | Primer paso |
|---|---|---|---|---|
| **Mesa de Servicios** (`mesa-servicios`) | 1.1.0 | 10 | 12 | `/mesa-servicios:start` |
| **Equipo del Product Owner** (`po-team`) | 1.0.0 | 8 | 9 | `/po-team:start` |
| **Dev Team** (`dev-team`) | 2.18.0 | 23 | 33 | `/dev-team:start` |
| **Marketing, Ventas y Publicidad** (`marketing`) | 1.0.0 | 8 | 10 | `/marketing:start` |
| **Arquitecto Senior Backend** (`senior-backend-architect`) | sin numero de version declarado | 1 | 1 | `/architect` |

FaaST tiene **cinco equipos de asistentes de inteligencia artificial**, cada uno con sus integrantes, sus reglas y su propio manual. Este es el **manual general**: explica que hace cada equipo, como se pasan el trabajo entre ellos en el flujo real de la empresa, como se instalan desde cero y que reglas comparten. El detalle de cada equipo esta en su propio manual (ver seccion 7). La parte tecnica (carpetas, configuracion, problemas de instalacion) esta al final, separada, en el **Anexo tecnico**.

---

## Indice


**Parte 1 — Para todas las personas**

1. Que es y para que sirve
2. Como se encadenan los equipos
3. Conoce a los equipos
4. Instalacion paso a paso, desde cero
5. Que equipo uso
6. Las reglas comunes a todos los equipos
7. Donde esta cada manual
8. Las palabras que vas a escuchar

**Parte 2 — Situaciones reales de punta a punta**

- Caso 1: Un cliente pide una funcionalidad nueva (de la cotizacion a la venta)
- Caso 2: Un cliente reporta un error
- Caso 3: Un incidente urgente en produccion

- Caso 4: Hay que corregir o revertir datos de un cliente
- Caso 5: Un proyecto nuevo completo
- Caso 6: Lanzar un modulo o contarle las mejoras a la cartera
- Caso 7: Una consulta tecnica puntual

**Parte 3 — Preguntas frecuentes y problemas comunes**

- Preguntas frecuentes
- Si algo no anda

**Anexo tecnico (para quien instala y configura)**

- A. Instalacion y actualizacion por linea de comandos
- B. Las carpetas de cada equipo
- C. Registro personal de carpetas
- D. Modelos de IA por integrante
- E. Problemas tecnicos comunes

---

# Parte 1 — Para todas las personas

## 1. Que es y para que sirve

Piensa en los equipos de agentes como **cinco areas de trabajo que se pueden contratar por horas**, cada una con especialistas que siguen el metodo de un profesional experimentado de su oficio. Tu les hablas en lenguaje normal, como le hablarias a un colega; ellos te presentan un plan, esperan tu "adelante" y hacen el trabajo, dejando todo documentado y con evidencia.

| Equipo | En una frase | Lo que entrega |
|---|---|---|
| **Mesa de Servicios** | Recibe todo lo que pide un cliente y lo deja como un requerimiento cerrado | Preguntas para el cliente, reproduccion de errores con video, maquetas, documento ejecutivo (BUG / PRY / CTZ) en PDF y Word, propuesta para el cliente y el ticket en el tablero |
| **Equipo del Product Owner** | Deja listo para desarrollo lo que cerro la Mesa y acompaña cada ticket hasta el VB del cliente | Historias y bugs en lenguaje de negocio con criterios y puntos, sprint con fechas de pase, tablero con avance e informe, revision rapida del PO en Demo con fotos, actas, informes y manuales de uso |
| **Dev Team** | Diseña, construye, prueba, asegura y publica el software | Historias de usuario, propuestas de diseño, codigo integrado, pruebas con fotos y video, revisiones de seguridad, solicitudes de pase a ambientes, documentacion |
| **Marketing** | Vende lo que se construyo, desglosado parte por parte | Mapa del producto con fichas, video keynote con un capitulo por parte, clips, deck de propuesta comercial, one-pager, panfletos y piezas para redes |
| **Arquitecto Senior Backend** | Un consultor tecnico experto para una consulta puntual | Diagnostico, revision de codigo, diseño de base de datos o de arquitectura, con recomendaciones concretas y, si lo pides, la implementacion |

**Lo que ganas:**

- Lo que pide un cliente llega a desarrollo **sin vacios**: nadie vuelve a preguntarle lo mismo.
- Cada ticket **se sigue hasta el visto bueno del cliente**, con fechas de pase y avance a la vista.
- Lo que se construye **se prueba con evidencia** (fotos y video) antes de llegar al cliente.
- Lo que se vende **es real**: se muestra el producto de verdad, con cifras que tienen fuente.
- Todo queda **registrado y trazable**: quien pidio que, que se decidio, que se entrego.

## 2. Como se encadenan los equipos

El flujo real de la empresa va de arriba hacia abajo. Cada equipo recoge el trabajo exactamente donde lo dejo el anterior.

```
flowchart TB C(["👤 <b>Cliente</b><br/>correo · reunión · audio<br/>chat · documento"]) M["<b>Mesa de Servicios</b><br/>recibe · pregunta · reproduce<br/>maqueta · documenta · cotiza"] P["<b>Equipo del Product Owner</b><br/>refina · planifica el sprint · sigue el tablero<br/>valida en Demo · consigue el VB del cliente"] D["<b>Dev Team</b><br/>diseña · construye · prueba<br/>asegura · publica"] K["<b>Marketing</b><br/>desglose parte por parte<br/>deck · video de lanzamiento · piezas"] A["<b>Arquitecto Senior Backend</b><br/>consultor puntual"] C --> M M -- "requerimiento cerrado<br/>en el tablero" --> P P -- "ticket listo:<br/>historia, sprint y fechas" --> D D -- "publicado en Demo:<br/>VB PO y VB cliente" --> P D -- "producto real: pantallas,<br/>memoria, diseño" --> K A -. "consulta técnica<br/>en cualquier momento" .-> D classDef equipo fill:#f1f7e9,stroke:#71af26,stroke-width:2px,color:#151515 classDef cliente fill:#151515,stroke:#151515,color:#ffffff classDef consultor fill:#ffffff,stroke:#4a4f45,stroke-dasharray:4 3,color:#151515 class M,P,D,K equipo class C cliente class A consultor
```

| Etapa | Equipo | Recibe | Entrega a |
|---|---|---|---|
| 1. Entender al cliente | Mesa de Servicios | Lo que llego del cliente, por cualquier canal | Un ticket cerrado en el tablero, con documento y evidencia adentro → **Equipo del PO** |
| 2. Dejar listo y planificar | Equipo del Product Owner | El ticket cerrado de la Mesa (no se re-entrevista al cliente; si falta algo, se le pregunta a la Mesa) | Historias y bugs listos para construir, con criterios, puntos, sprint y fechas de pase → **Dev Team**, siempre como ticket en el tablero |
| 3. Construir | Dev Team | El ticket ya refinado | Funcionalidad probada, integrada y publicada en un ambiente |
| 4. Validar y cerrar con el cliente | Equipo del Product Owner | Lo publicado en Demo o en el Preprod del cliente | VB PO u Observado con fotos, acta de aceptacion y el VB del cliente registrado |
| 5. Vender | Marketing | Lo que el Dev Team y la Mesa ya tienen (memoria, diseño, maquetas, ambiente demo). **Solo lo lee** | Piezas comerciales aprobadas por ti |

| Etapa | Equipo | Recibe | Entrega a |
|---|---|---|---|
| En paralelo | Arquitecto Senior Backend | Una pregunta tecnica concreta | Diagnostico y recomendacion |

> **Si no usas el Equipo del PO**, el flujo sigue funcionando: el dueño del producto del Dev Team toma directamente el ticket cerrado de la Mesa y lo convierte en historias ( `/dev-team:refine` ). El Equipo del PO agrega el seguimiento por sprint con fechas de pase, la revision rapida en Demo y el cierre con el cliente.

> **Las cotizaciones esperan.** Si lo que pidio el cliente hay que cotizarlo, la Mesa deja el ticket **en espera de aprobacion comercial**. Recien cuando comercial aprueba (y queda registrado quien y cuando), pasa a ser trabajo para el Equipo del PO y el Dev Team.

---

## 3. Conoce a los equipos

### Mesa de Servicios — "la que escucha al cliente"

10 integrantes. **Tu hablas con el cliente; el equipo prepara las preguntas, las maquetas, los documentos y los tickets.** El jefe de mesa reparte el trabajo; recepcion guarda tal cual lo que llego; el analista prepara rondas cortas de preguntas (como mucho siete por ronda, como mucho tres rondas) despues de leer la memoria del negocio ( `faast-brain` ) para no preguntar lo obvio; el probador reproduce los errores grabando todo (**solo prueba, nunca busca la causa**); el de los datos mira y arma scripts (**nunca ejecuta nada**); el maquetador dibuja bocetos navegables; el redactor escribe el documento ejecutivo; el registrador sube el ticket; el curador guarda lo aprendido; y setup prepara la computadora.

### Equipo del Product Owner — "el que deja listo y acompaña hasta el VB"

8 integrantes. **Tu decides que entra y cuando; el equipo prepara.** El coordinador reparte el trabajo y te muestra el plan antes de tocar el tablero; el refinador convierte lo que dejo la Mesa en historias de negocio (entre 3 y 8 criterios con los casos limite, pendientes a la vista, definicion de hecho y puntos) y registra los bugs **solo si se reprodujeron**; el planificador arma el sprint por capacidad, con un orden que puedes ver, deja afuera lo que no esta listo y fija las fechas de Demo, Preprod y VB cliente, y al final hace la revision y la retrospectiva; el de seguimiento lee el tablero y arma el panel y el informe (**solo mira**); el revisor rapido valida en Demo o Preprod con una foto por criterio, reproduce los errores reportados y vuelve a probar los arreglos (**no reemplaza a QA**); el documentador prepara actas, especificaciones e informes; el manualista escribe un manual para QA y otro para cualquier persona; y setup prepara la computadora y el tablero. Entrega el trabajo al equipo de desarrollo **solo como tickets**.

### Dev Team — "la empresa de software en miniatura"

23 integrantes. Un dueño del producto que escribe las historias en lenguaje de negocio, un arquitecto, un estudio de diseño completo (direccion, experiencia, grafica, animacion, 3D y maquetacion), un jefe de equipo que coordina y es el unico que integra cambios, programadores de servicios y de pantallas, un encargado de bases de datos, un equipo de pruebas (jefe + probador de pantallas + probador de servicios), un equipo de seguridad (jefe + hacker etico + seguridad del codigo + seguridad de plataforma), un encargado de publicaciones (pases), infraestructura, un documentador y setup. **Nada se integra sin la aprobacion de pruebas, con evidencia.**

### Marketing, Ventas y Publicidad — "el que vende el producto real"

8 integrantes: director de marketing (dirige y es el unico que delega), investigador de mercado, analista de producto, estratega comercial, redactor publicitario, diseñador de presentaciones, director de motion (videos) y diseñador grafico. **Nunca hace una presentacion generica:** primero arma el **mapa del producto** con una **ficha por parte** (que es, que problema resuelve y cuanto cuesta, como funciona con pantallas reales, que gana el cliente con fuente, antes/despues y comparacion con la competencia). Tu apruebas el mapa y las fichas; recien ahi se producen las piezas, y cada pieza tambien se aprueba.

### Arquitecto Senior Backend — "el consultor"

1 integrante con 1 comando. Un experto en backend (.NET, Python, Java, Node.js, C++, Go, Rust y bases de datos relacionales y NoSQL) que trabaja en cuatro fases: **entender** (si falta algo critico, hace una sola pregunta), **analizar**, **recomendar** (lo mas critico primero, con el por que y la solucion concreta) e **implementar** solo si lo necesitas. No tiene carpeta de trabajo ni tablero: es una consulta de una vez. Responde en el idioma en que le escribes.

---

## 4. Instalacion paso a paso, desde cero

Esta seccion es para quien nunca instalo nada. Toma unos 15 minutos la primera vez. Si alguien de tu area ya lo dejo instalado en tu computadora, salta al Paso 7.

### Paso 1 — Que es Claude Code y donde se usa

**Claude Code** es la aplicacion donde viven los equipos. Es una conversacion con Claude, como un chat, pero que puede trabajar con los archivos de tu computadora. Los equipos de FaaST se instalan dentro de ella como "plugins" (complementos).

Se puede usar en tres lugares; elige el que te sea mas comodo (los comandos son los mismos):

| Donde | Como se ve | Para quien |
|---|---|---|
| **La terminal** (Terminal en Mac, PowerShell o Terminal en Windows) | Una ventana de texto donde escribes y Claude responde | La forma mas completa; la que usan estos manuales en sus ejemplos |

| Donde | Como se ve | Para quien |
|---|---|---|
| **VS Code** (editor gratuito) con la extension **Claude Code** | Un panel de conversacion al costado del editor | Si ya usas VS Code o quieres ver los archivos mientras trabajas |
| **La aplicacion de escritorio de Claude** | La app de Claude, en su seccion de codigo | Si prefieres una aplicacion con ventanas y botones |

> **Como abrir la terminal.** En Mac: presiona `Cmd + Espacio`, escribe `Terminal` y presiona Enter. En Windows: menu Inicio, escribe `PowerShell` y abrelo.

### Paso 2 — Instalar Claude Code e iniciar sesion

Necesitas una **cuenta de Claude** habilitada para Claude Code (pidesela a tu administrador si no la tienes). Luego, en la terminal, copia y pega **una** de estas lineas y presiona Enter:

```
# Mac o Linux
curl -fsSL https://claude.ai/install.sh | bash

# Windows (PowerShell)
irm https://claude.ai/install.ps1 | iex
```

**Que ves:** unas lineas de progreso y, al final, un mensaje de que quedo instalado. Para comprobarlo, escribe:

```
claude --version
```

→ responde con un numero de version. Si dice "command not found" o "no se reconoce", cierra la terminal, abrela de nuevo y repite.

Ahora inicia sesion:

```
claude
```

**Que ves la primera vez:** te pide elegir como iniciar sesion y abre el navegador para que entres con tu cuenta de Claude. Al volver a la terminal ya estas dentro de una conversacion. Para salir, escribe `/exit`.

> **Requisitos que conviene tener** (si no los tienes, el integrante **setup** de cada equipo te los ofrece con una sola confirmacion): `git`, `python3` (lo usan los controles automaticos de los equipos) y `Node.js`. Para video de Marketing, Node 22 o superior y FFmpeg.

### Paso 3 — Abrir la carpeta de trabajo

Los equipos trabajan **en la carpeta donde abres Claude Code**. Por eso, antes de escribir `claude`, te paras en la carpeta correcta:

```
cd ~/Documents/Trabajo/Faast/Proyectos/Backoffice
claude
```

| Equipo | Donde abrirlo |
|---|---|
| Mesa de Servicios | En la **carpeta de la mesa** (la primera vez te ayuda a crearla) |
| Equipo del Product Owner | En la **carpeta del proyecto** o en tu **carpeta de PO**, nunca dentro de un repositorio (la primera vez te ayuda a elegirla) |
| Dev Team | En la **carpeta del proyecto**, la que contiene todos sus repositorios |
| Marketing | En la **carpeta del proyecto** que se va a vender |
| Arquitecto Senior Backend | En el **repositorio** que quieres revisar |

**Que ves la primera vez en una carpeta:** Claude Code pregunta si confias en los archivos de esa carpeta. Si es tu carpeta de trabajo, responde que si.

> En VS Code: menu **Archivo → Abrir carpeta** y luego abres el panel de Claude Code. En la app de escritorio: eliges la carpeta al iniciar la conversacion de codigo.

### Paso 4 — Agregar el marketplace de FaaST (una sola vez)

El **marketplace** es la "tienda" interna de FaaST donde estan publicados los cinco equipos. Se agrega una sola vez por computadora. En la terminal (fuera de Claude Code):

```
claude plugin marketplace add faast-app/faast-claude-marketplace
```

**Que ves:** un mensaje confirmando que el marketplace `faast-marketplace` quedo agregado.

> Tambien se puede hacer **dentro** de una conversacion de Claude Code, escribiendo `/plugin marketplace add faast-app/faast-claude-marketplace`. Si aparece un error de acceso, tu cuenta de GitHub necesita acceso a la organizacion `faast-app`: pidelo a tu administrador.

### Paso 5 — Instalar los equipos que vas a usar

Instala solo los que necesitas (ver seccion 5). Uno por linea:

```
claude plugin install mesa-servicios@faast-marketplace
claude plugin install po-team@faast-marketplace
claude plugin install dev-team@faast-marketplace
claude plugin install marketing@faast-marketplace
claude plugin install senior-backend-architect@faast-marketplace
```

**Que ves:** por cada uno, un mensaje de que quedo instalado. Dentro de Claude Code tambien puedes escribir `/plugin` y elegirlos de un menu.

### Paso 6 — Reiniciar y verificar

Los equipos se cargan **al iniciar** Claude Code. Si tenias una conversacion abierta, cierrala (`/exit`) y vuelve a abrirla (`claude`). Para comprobar que quedaron instalados:

```
claude plugin list
```

→ ves la lista de plugins instalados. Deben aparecer los que instalaste, del marketplace `faast-marketplace`.

Dentro de la conversacion, escribe `/` y empieza a tipear el nombre del equipo (`/dev-team`, `/mesa-servicios`, `/po-team`, `/marketing`, `/architect`): aparecen sus comandos sugeridos.

### Paso 7 — El primer paso en cada equipo

Cada equipo tiene **un solo comando de entrada** que detecta en que situacion estas y te guia. No necesitas memorizar nada mas.

#### Mesa de Servicios

```
/mesa-servicios:start
```

**Que ves la primera vez:** el integrante setup te pregunta, en una sola ronda, lo minimo (tablero y proyecto donde van los tickets, clientes, productos, memoria del negocio, repositorio de evidencia y ambientes), revisa la computadora e instala lo que falte **con tu confirmacion**. Luego te ofrece crear la memoria del negocio (`/mesa-servicios:brain init`) y te muestra como recibir la primera solicitud. **Las veces siguientes:** un resumen corto (solicitudes en curso, esperando cliente, pendientes de aprobacion comercial) y la siguiente accion sugerida.

#### Equipo del Product Owner

```
/po-team:start
```

**Que ves la primera vez:** el integrante setup revisa la computadora e instala lo que falte **con tu confirmacion**, te propone donde guardar la carpeta del equipo (en la carpeta del proyecto o tu carpeta de PO, nunca dentro de un repositorio) y te pregunta en una sola ronda el tablero (GitHub o Azure DevOps), que estado

corresponde a cada etapa del ciclo de pase, los ambientes con su clase (de pruebas o productivo), los clientes y la capacidad del sprint. Si todavia no tienes tablero, te ofrece crearlo (`/po-team:tablero crear-backlog`). Las claves de las cuentas de prueba las escribes tu en un archivo, nunca en la conversacion (ver su manual). **Las veces siguientes:** el sprint en curso por etapa, el proximo hito, tus pendientes y que conviene hacer ahora. Puedes escribir la necesidad en la misma linea: `/po-team:start la mesa dejo el #131, hay que pasarlo a HU`.

**Dev Team**

```
/dev-team:start
```

**Que ves la primera vez:** te pregunta si quieres **crear un proyecto nuevo** (desde una idea o un documento) o **heredar uno existente**. En ambos casos, setup revisa la computadora y te pide una sola confirmacion para instalar lo que falte. **Las veces siguientes:** el estado del proyecto y lo que conviene hacer ahora. Puedes escribir la necesidad en la misma linea: `/dev-team:start los analistas necesitan filtrar cobranzas por fecha`.

**Marketing**

```
/marketing:start
```

**Que ves la primera vez:** te propone donde guardar su carpeta de trabajo (en la carpeta del proyecto, nunca dentro de un repositorio) y te pregunta en un solo mensaje que producto se vende, donde estan sus fuentes (Dev Team, Mesa, prototipos), si hay un ambiente demo, a quien se le vende y si hay manual de marca. Revisa las herramientas de video y documentos, te ofrece cargar la marca (`/marketing:marca`) y la base de tendencias (`/marketing:tendencias`), y te muestra como empezar. **Las veces siguientes:** campañas en curso, piezas esperando tu aprobacion y que sigue.

**Arquitecto Senior Backend**

```
/architect mi endpoint de .NET 8 tarda 30 segundos en responder, ayudame a diagnosticarlo
```

**Que ves:** el analisis en cuatro fases. Si le falta algo critico, primero te hace una sola pregunta. (Si otro plugin tuviera un comando con el mismo nombre, se escribe `/senior-backend-architect:architect`.)

## Paso 8 — Mantenerlos al dia

Cuando se publiquen novedades, en la terminal:

```
claude plugin marketplace update faast-marketplace
claude plugin update dev-team@faast-marketplace
claude plugin update mesa-servicios@faast-marketplace
claude plugin update po-team@faast-marketplace
claude plugin update marketing@faast-marketplace
claude plugin update senior-backend-architect@faast-marketplace
```

Y reinicia Claude Code. El Dev Team y el Equipo del PO ademas **te avisan solos**: al abrir una sesion comparan tu version con la publicada y te preguntan si quieres actualizar.

---

## 5. Que equipo uso

| Situacion | Equipo | Escribes |
|---|---|---|
| Llego un correo, una reunion grabada o un chat de un cliente con un pedido | Mesa de Servicios | `/mesa-servicios:recibir {archivo o texto} --cliente ACME` |
| El cliente dice que algo falla | Mesa de Servicios (reproduce y documenta) → Equipo del PO (lo deja listo y lo sigue) → Dev Team (corrige) | `/mesa-servicios:reproducir {ID}` |
| Hay que cotizar algo para un cliente | Mesa de Servicios | `/mesa-servicios:start necesito cotizar ...` |
| Hay que corregir o revertir datos | Mesa de Servicios (arma los scripts, no los ejecuta) | `/mesa-servicios:sql script {ID}` |
| La Mesa dejo un requerimiento cerrado y hay que pasarlo a historias | Equipo del PO | `/po-team:refinar #{ticket}` |
| Armar el sprint con sus fechas de Demo, Preprod y VB del cliente | Equipo del PO | `/po-team:sprint planificar {N}` |
| Saber como va el sprint (tablero e informe del dia) | Equipo del PO | `/po-team:tablero diario` |
| Revisar en Demo lo que llego y dar el VB PO | Equipo del PO | `/po-team:validar --lote sprint {N} --ambiente {demo}` |
| Me reportaron un error y quiero confirmarlo antes de registrarlo | Equipo del PO | `/po-team:validar --reproducir "{lo que reportaron}"` |

| Situacion | Equipo | Escribes |
|---|---|---|
| El acta para el cliente o registrar su VB | Equipo del PO | `/po-team:documento aceptacion #{n}` · `/po-team:refinar --vb-cliente #{n}` |
| Un requerimiento ya cerrado hay que construirlo, sin usar el Equipo del PO | Dev Team | `/dev-team:refine {pedido o ticket}` |
| Tengo una idea o un documento y quiero un proyecto nuevo | Dev Team | `/dev-team:new-project {idea o documento}` |
| Tengo una propuesta por fases y quiero el tablero completo armado | Dev Team | `/dev-team:project crear {documento}` |
| Quiero ver como se vera una pantalla antes de construirla | Dev Team (estudio de diseño) | `/dev-team:design {lo que necesitas}` |
| Un error ya registrado hay que corregirlo | Dev Team | `/dev-team:bug {ID o descripcion}` |
| Un incidente urgente en produccion | Dev Team | `/dev-team:hotfix {incidente} --ambiente prod` |
| Hay que publicar una version en un ambiente (pase) | Dev Team | `/dev-team:pase {ambiente}` |
| Necesito una revision de seguridad o un pentest | Dev Team (equipo de seguridad) | `/dev-team:security-audit` · `/dev-team:pentest {alcance}` |
| Un documento de negocio presentable (HU, sprint, acta de entrega) | Dev Team (dueño del producto) | `/dev-team:doc {tipo} {ref}` |
| Quiero vender el producto a un ejecutivo | Marketing | `/marketing:propuesta-comercial "{producto} para {publico}"` |
| Un video de lanzamiento o clips por parte | Marketing | `/marketing:video keynote` · `/marketing:video clips` |
| Un one-pager, panfleto o piezas para redes | Marketing | `/marketing:pieza {tipo}` |
| Datos de mercado o competencia con fuente | Marketing | `/marketing:investigar {tema}` |
| Una duda tecnica o una revision de codigo puntual, fuera de un proyecto del Dev Team | Arquitecto Senior Backend | `/architect {pregunta}` |

| Situacion | Equipo | Escribes |
|---|---|---|
| No se por donde empezar | El equipo mas cercano a tu pedido | `/mesa-servicios:start` · `/po-team:start` · `/dev-team:start` · `/marketing:start` |

> **Regla practica:** si viene **del cliente** y todavia no esta claro, empieza en la **Mesa**. Si ya esta claro y hay que **dejarlo listo, planificarlo y seguirlo hasta el VB del cliente**, es el **Equipo del PO**. Si hay que **construirlo**, es **Dev Team**. Si ya existe y hay que **venderlo**, es **Marketing**. Si es una **pregunta tecnica suelta**, es el **Arquitecto**.

---

## 6. Las reglas comunes a todos los equipos

Estas reglas estan siempre activas en todos los equipos. No hay que pedirlas. Cada manual detalla como las aplica su equipo.

1. **Plan primero.** Antes de hacer algo que cambie algo (construir, corregir, mandar preguntas al cliente, maquetar, armar scripts, subir un ticket, producir una pieza), el equipo te muestra el plan y espera tu "adelante". Lo que solo mira (resumir, buscar, consultar) corre directo. El Arquitecto Senior recomienda antes de implementar, e implementa solo si se lo pides.
2. **Nada se inventa.** Lo que el cliente no respondio queda como **pendiente explicito**, no como "lo que parece razonable". En Marketing, cada cifra, comparacion o afirmacion lleva **fuente y fecha**; lo que no tiene respaldo se marca como **supuesto**. Las versiones, los responsables y los destinatarios salen de la configuracion o se te preguntan; nunca se escriben a mano.
3. **Tus pedidos valen aunque pasen por la sesion o por el lider del equipo.** Si escribes "dile al de los datos que revise la operacion 10432" o "que pruebas revise la historia 42", el integrante lo hace como si se lo hubieras dicho a el. **La unica excepcion:** cuando Claude Code te pide aprobar un permiso en pantalla, eso lo apruebas tu; nadie puede darlo por autorizado.
4. **Nada del equipo se guarda dentro de los repositorios.** Tareas, notas, fotos, videos, documentos, campañas, metricas y accesos viven en la **carpeta de trabajo de cada equipo**, al lado de los repositorios, nunca adentro. **Al repositorio solo va lo que es del producto:** codigo, migraciones de base de datos y documentacion tecnica (y la automatizacion de publicacion). Un control automatico impide guardar la carpeta de un equipo dentro de un repositorio; si una carpeta antigua quedo adentro, el equipo te avisa y te propone moverla.
5. **Ambientes de pruebas y productivos.** Cada ambiente esta declarado como **de pruebas** o **productivo**; nunca se supone por el nombre (si aparece uno sin declarar, te preguntan una vez y lo recuerdan).
    - En un ambiente **de pruebas**, aprobar, girar o pagar son acciones de prueba normales: el equipo las hace sin pedirte OK una por una.

- En un ambiente **productivo**, cada accion delicada necesita tu autorizacion; en la Mesa, cada corrida lleva su autorizacion registrada (quien, cuando, para que caso) y el equipo se detiene antes de cada paso que cambie algo. El revisor rapido del Equipo del PO, sin autorizacion para esa accion, solo mira.
- **Cambiar parametros o datos de una base de datos siempre lleva tu OK**, en cualquier ambiente. Y la Mesa nunca ejecuta scripts: los arma para quien esta autorizado.
- Marketing captura el producto **solo en ambientes de pruebas o demo, con datos ficticios**.

6. **Los pendientes no se pierden.** Si un equipo te hizo una pregunta y no alcanzaste a responder, queda registrada y aparece en una lista corta al abrir la siguiente sesion, hasta que la resuelvas.

7. **Toda prueba deja evidencia, y el video se ve a tamaño real.** Los probadores (Dev Team y Mesa) graban un video que ocupa toda la pantalla y se lee nitido, con un **capitulo por paso**, en **MP4** (abre en Windows, Mac, Teams y el navegador), mas fotos de cada paso con lo importante resaltado. En el ticket aparece una **portada** que abre el video completo; la evidencia se ve **dentro** del ticket, no como un enlace aparte, y nunca se mezcla con el codigo. Si un video salio chico o borroso, se vuelve a grabar: un video que no se lee no cuenta como evidencia. El revisor rapido del Equipo del PO deja una foto verificada por criterio y graba video cuando reproduce un error o vuelve a probar un arreglo.

8. **Confidencialidad.** Sin contraseñas, claves ni datos personales en documentos, tickets, memorias ni piezas. Se habla de empresas y roles, no de personas. Los accesos viven en archivos protegidos que nunca llegan a un repositorio. Las grabaciones de reuniones se transcriben **en tu computadora**. Las piezas de Marketing no llevan datos de clientes ni informacion interna, ni logos de clientes sin autorizacion, y los precios solo los entrega comercial. En seguridad, todo lo sensible sale tachado de los informes.

9. **Cada integrante usa el "cerebro" (modelo de IA) del tamaño justo.** Por defecto, casi todos usan uno intermedio (`sonnet`); en el Dev Team, el que prepara la oficina y el documentador usan el economico (`haiku`) y no se pueden cambiar, y en el Equipo del PO, el que prepara la oficina tambien usa `haiku` fijo. Si un trabajo lo amerita, se puede subir un integrante a uno mas potente (`opus`) por proyecto o para todos tus proyectos (ver Anexo D). Mas potente piensa mejor, pero cuesta mas.

10. **Las personas hablan con las personas.** Ningun equipo contacta al cliente por su cuenta ni envia correos solo: te deja las preguntas, el correo o la propuesta listos y **tu los envias**.

11. **Solo el lider de cada equipo reparte trabajo entre los integrantes.** Los demas hacen su trabajo directo. Esto mantiene el orden y evita que el costo se dispare. La Mesa y el Equipo del PO le pasan el trabajo al Dev Team **como ticket en el tablero**, nunca invocando a sus integrantes.

12. **Un error se confirma antes de registrarlo.** La Mesa, el Equipo del PO y el Dev Team reproducen el error con evidencia (**REPRODUCIDO**, **NO REPRODUCIDO** o **BLOQUEADO**) antes de registrarlo o de hablar de la correccion. Un arreglo se vuelve a probar con el mismo recorrido y fotos **antes / despues**, y nadie cierra un ticket sin tu confirmacion.

13. **Hablas con uno solo, desde donde estes.** Tu unica conversacion es la principal. Ningun integrante te pide "escribeme a mi directamente": si necesita tu autorizacion, el coordinador te la pregunta y se la pasa con tus palabras textuales, y el integrante comprueba en el registro de tus mensajes que de verdad lo dijiste. Funciona igual desde el telefono o con Remote Control.

14. **Si el sistema no deja hacer algo, no se rodea.** Cuando Claude Code deniega una accion a un integrante, este sigue con lo demas y te lo reporta con el comando exacto; tu decides. En el Dev Team, `/dev-team:setup permisos` deja pre-aprobado lo rutinario y seguro.

15. **El diseño hereda de la libreria de FaaST y el equipo aprende** (Dev Team). Las propuestas de diseño se arman con los colores, la tipografia y los componentes de la libreria oficial (`@faast-app`); y lo que corriges, eliges o rechazas queda como leccion para la proxima vez.

---

## 7. Donde esta cada manual

Todos estan en el repositorio del marketplace (github.com/faast-app/faast-claude-marketplace):

| Manual | Archivo | Version para imprimir o compartir |
|---|---|---|
| Este manual general | `GUIDE.md` | `GUIDE.pdf` |
| Mesa de Servicios — Manual de Usuario | `plugins/mesa-servicios/GUIDE.md` | `plugins/mesa-servicios/GUIDE.pdf` |
| Mesa de Servicios en Claude web (version simplificada, sin instalar nada) | `plugins/mesa-servicios/claudeweb/INSTRUCCIONES.md` | `plugins/mesa-servicios/claudeweb/INSTRUCCIONES.pdf` |
| Equipo del Product Owner — Manual de Usuario | `plugins/po-team/GUIDE.md` | `plugins/po-team/GUIDE.pdf` |
| Dev Team — Manual de Usuario | `plugins/dev-team/GUIDE.md` | `plugins/dev-team/GUIDE.pdf` |
| Marketing — Manual de Usuario | `plugins/marketing/GUIDE.md` | `plugins/marketing/GUIDE.pdf` |
| Marketing — resumen | `plugins/marketing/README.md` | — |
| Arquitecto Senior Backend | `plugins/senior-backend-architect/README.md` | — |

Cada manual de equipo sigue la misma estructura que este: Parte 1 para todas las personas, Parte 2 con situaciones reales paso a paso, Parte 3 con preguntas frecuentes y un Anexo tecnico.

---

## 8. Las palabras que vas a escuchar

| Palabra | Que significa |
|---|---|
| **Plugin** | Un complemento que se instala en Claude Code. Cada equipo es un plugin |
| **Marketplace** | La "tienda" interna de FaaST donde se publican los plugins (`faast-marketplace`) |
| **Comando** | Lo que escribes empezando con `/`, como `/dev-team:start`. El nombre antes de los dos puntos es el equipo |
| **Sesion** | Una conversacion abierta de Claude Code. Al abrirla se cargan los equipos y sus pendientes |
| **Solicitud** | Todo lo que llega de un cliente y trabaja la Mesa. Se convierte en **BUG** (algo falla), **PRY** (algo nuevo) o **CTZ** (hay que cotizarlo) |
| **Correlativo** | El numero unico de cada documento de la Mesa: `BUG-2026-003`, `PRY-2026-014`, `CTZ-2026-007` |
| **Alcance** | Para quien es: un **cliente** especifico o **transversal** (todos los que usan el producto) |
| **Ticket / tablero** | La tarjeta con todo adentro y el lugar donde viven (GitHub o Azure DevOps). Tambien se le dice **backlog** |
| **Historia de usuario (HU)** | Una necesidad escrita desde quien la usa, con criterios "Dado… cuando… entonces…" que dicen cuando esta lista |
| **Reproducir** | Hacer en el sistema real los pasos que el cliente dice que fallan, para confirmarlo con evidencia. Veredicto: **REPRODUCIDO**, **NO REPRODUCIDO** o **BLOQUEADO** |
| **Evidencia** | Fotos, video y registros que demuestran lo que se probo |
| **Maqueta / prototipo** | En la Mesa, un boceto navegable para validar la idea. En el Dev Team, un prototipo de alta fidelidad que se recorre como el producto real |
| **Pase** | Publicar una version en un ambiente formal (certificacion, puente, demo, preprod, produccion de clientes). Lleva una solicitud formal |
| **Ciclo de pase** | El camino de cada ticket en el Equipo del PO: Desarrollo → Validacion QA → Pase a Demo → Validacion Demo / Preprod → VB cliente. Si algo queda Observado, vuelve a Desarrollo |
| **Definition of Ready (lista para el sprint)** | Lo que una historia debe cumplir para entrar a un sprint: criterios claros, pendientes resueltos, tamaño razonable, entrega definida y quien da el VB. Lo que no la cumple se propone sacar |

| Palabra | Que significa |
|---|---|
| **VB PO** | El visto bueno del PO despues de su revision rapida en Demo o Preprod, con una foto por criterio. No reemplaza la prueba de QA |
| **Observado** | Lo que no cumple un criterio en la revision del PO. Vuelve a desarrollo con la explicacion y la foto dentro del ticket |
| **VB cliente** | El visto bueno del cliente (quien, cuando y por que medio: correo, acta firmada, reunion). Cierra el ciclo; cerrar el ticket es decision tuya |
| **Ambiente de pruebas / productivo** | Cada copia del sistema esta declarada como de pruebas (las acciones son pruebas normales) o productiva (dinero y datos reales; todo cambio con autorizacion) |
| **Desglose / mapa del producto** | En Marketing, el producto separado en sus partes, con una ficha por parte. Es la base de toda pieza |
| **Memoria** | Lo que cada equipo aprende y guarda: `faast-brain` en la Mesa (el negocio, que el Equipo del PO tambien lee), la wiki en el Dev Team (el proyecto) |
| **Carpeta de trabajo** | Donde cada equipo guarda lo suyo, fuera de los repositorios (ver Anexo B) |
| **Repositorio** | La carpeta donde vive el codigo de un componente, con su historial de cambios |
| **Pendiente** | Algo que espera tu respuesta o tu decision. Se te recuerda al abrir cada sesion |
| **Plan primero** | La regla de que nadie cambia nada sin mostrarte antes el plan y esperar tu "adelante" |

---

# Parte 2 — Situaciones reales de punta a punta

Cada caso cruza equipos. Muestra **que escribes exactamente**, quien hace que y que recibes. Los ejemplos son de un cliente ficticio ("ACME"). El detalle de cada paso esta en el manual del equipo correspondiente.

---

## Caso 1 — Un cliente pide una funcionalidad nueva (de la cotizacion a la venta)

**Situacion:** ACME escribe: "queremos que el sistema se integre con nuestro banco para conciliar los pagos automaticamente". Hay que entenderlo, cotizarlo, construirlo y, si sirve a otros clientes, venderlo.

**Etapa 1 — La Mesa lo entiende y lo cotiza**

1. Recibir el correo:

   ```
   /mesa-servicios:recibir correo-acme-banco.eml --cliente ACME
   ```

   → `SOL-2026-03-11-1 — Conciliacion automatica con el banco — ACME — tipo preliminar: cotizacion`, con lo que se entendio y lo que falta.
2. Cerrar las dudas con el cliente:

   ```
   /mesa-servicios:analizar SOL-2026-03-11-1
   ```

   → una ronda de hasta siete preguntas con opciones concretas. **Tu** la llevas a ACME y luego cargas las respuestas con `--respuestas "..."`. Al cerrar: `CERRADA · Tipo: CTZ · Alcance: Cliente ACME`.
3. Que el cliente vea la idea:

   ```
   /mesa-servicios:prototipo SOL-2026-03-11-1
   ```

   → una maqueta tipo boceto, navegable, con preguntas de validacion por pantalla.
4. El documento y la propuesta:

   ```
   /mesa-servicios:documento SOL-2026-03-11-1
   ```

   → `CTZ-2026-007` en PDF y Word, mas la **Propuesta para el cliente** con la valorizacion en blanco para que la complete comercial.
5. El ticket, en espera:

```
/mesa-servicios:ticket CTZ-2026-007
```

→ el ticket queda en el tablero **en espera de aprobacion comercial**. Todavia no es trabajo.

6. Comercial aprueba:

```
/mesa-servicios:ticket aprobar CTZ-2026-007 "aprobado por Gerencia Comercial el 20/03/2026, propuesta v1"
```

→ el ticket pasa a la columna de trabajo. Lo aprendido se guarda con `/mesa-servicios:brain ingest CTZ-2026-007`.

**Etapa 2 — El Equipo del PO lo deja listo y lo planifica**

7. Convertir el ticket de la Mesa en historias:

```
/po-team:refinar #212
```

(`#212` es el ticket que registro la Mesa para `CTZ-2026-007`). → el refinador parte del documento de la Mesa (**no se re-entrevista al cliente**; si falta algo, deja la pregunta a la Mesa) y te muestra el plan. Con tu OK quedan, por ejemplo, `#58 Conciliar los pagos del banco automaticamente` y `#59 Ver las diferencias de la conciliacion`, con 3 a 8 criterios "Dado… cuando… entonces…", pendientes, definicion de hecho y puntos.

8. Armar el sprint:

```
/po-team:sprint planificar 14
```

→ que entra, que queda afuera y por que, y las fechas de Demo, Preprod de ACME y VB cliente. Con tu OK, el sprint y las fechas quedan en el tablero: **el Dev Team recibe el trabajo como ticket**.

**Etapa 3 — El Dev Team lo construye**

9. Traer las historias ya refinadas:

```
/dev-team:sync pull
```

→ el Dev Team trabaja sobre esas mismas historias (en sus ejemplos, la `#58` aparece como `HU-58`). Sin Equipo del PO, aqui el dueño del producto del Dev Team las escribe con `/dev-team:refine ticket de la Mesa CTZ-2026-007 ...`.

10. Si hay pantallas nuevas, el estudio de diseño te muestra caminos reales antes de programar:

```
/dev-team:design pantalla HU-58
```

11. Plan y construccion:

```
/dev-team:assign-task
```

→ el jefe de equipo te presenta el plan (que, quien, riesgos) y espera tu OK. Los programadores trabajan en paralelo.

12. Pruebas, con video y fotos por criterio:

```
/dev-team:e2e HU-58
```

→ veredicto **APROBADA** o **RECHAZADA**, con la evidencia dentro del ticket. Como toca pagos, pasa tambien por seguridad (`/dev-team:security-audit`). Recien entonces el jefe de equipo integra.

13. Publicar en el ambiente del cliente:

```
/dev-team:pase certificacion — Conciliacion bancaria v1.0.0
```

→ la carpeta del pase (solicitud en PDF y Word, scripts revisados, correo listo). **Tu lo envias.**

**Etapa 4 — El Equipo del PO lo valida y consigue el VB del cliente**

14. Revision rapida del PO en Demo y en el Preprod de ACME:

```
/po-team:validar --lote sprint 14 --ambiente preprod-acme
```

→ **VB PO** u **Observado** por ticket, con una foto por criterio (lo observado vuelve a desarrollo, bien explicado).

15. El acta para el cliente y su VB:

```
/po-team:documento aceptacion #58
/po-team:refinar --vb-cliente #58
```

→ el acta en PDF y Word; cuando ACME da su conformidad, queda registrado quien, cuando y por que medio. (Sin Equipo del PO, el acta la prepara el Dev Team con `/dev-team:doc aceptacion HU-58`.)

**Etapa 5 — Marketing lo vende** (si la funcionalidad le sirve a otros clientes)

16.
```
/marketing:propuesta-comercial "conciliacion bancaria automatica para gerentes de finanzas de la cartera" --pais CL
```

→ Marketing lee lo que ya tienen el Dev Team y la Mesa (sin escribir en sus carpetas), arma el **mapa del producto** y una **ficha por parte**, y espera tu aprobacion. Con mapa y fichas aprobados produce el **video**

**keynote** (un capitulo por parte), los **clips por parte**, el **deck** y el **one-pager**. Cada pieza pasa por tu aprobacion.

**Recibes:** un pedido de cliente convertido en cotizacion aprobada, historias listas y planificadas, funcionalidad construida y probada con video, publicada con su pase, con el VB del cliente registrado en su acta y lista para venderse.

---

## Caso 2 — Un cliente reporta un error

**Situacion:** ACME dice: "a veces se duplican las operaciones cuando paso a la pagina 2".

1. **La Mesa lo recibe y cierra los pasos** con el cliente (`/mesa-servicios:recibir` y `/mesa-servicios:analizar`).
2. **La Mesa lo reproduce**, grabando todo:

   ```
   /mesa-servicios:reproducir SOL-2026-03-11-2
   ```

   → `Veredicto: REPRODUCIDO (siempre) en pruebas · 3 fotos, video 18 s`. El probador **nunca dice por que falla**: solo demuestra que falla. Si **no** se reproduce, no se inventa el error: te dice que se probo y que falta para volver a intentar.
3. **Documento y ticket:**

   ```
   /mesa-servicios:documento SOL-2026-03-11-2
   /mesa-servicios:ticket BUG-2026-003
   ```

   → el ticket queda en el tablero con el video y las fotos dentro.
4. **El Equipo del PO lo deja listo para desarrollo:**

   ```
   /po-team:refinar #240
   ```

   → como la Mesa ya lo reprodujo, el refinador registra el bug en lenguaje de negocio (pasos como usuario, esperado contra obtenido, ambiente, severidad sugerida y la evidencia de la Mesa dentro) y, con tu OK, lo pone en el sprint (`/po-team:sprint replanificar` si es urgente).
5. **El Dev Team lo toma:**

   ```
   /dev-team:bug #240
   ```

   → aunque el ticket ya trae la reproduccion de la Mesa, la regla del Dev Team es **reproducir antes de corregir**, en su ambiente y anotando la version instalada. Con el veredicto REPRODUCIDO, el jefe de equipo te presenta el plan de correccion y espera tu OK.

6. **Correccion y revalidacion:** el programador corrige; pruebas escribe una prueba automatica para que el error no vuelva y **revalida** el mismo recorrido con la correccion instalada, con un par de evidencia **antes / despues**.
7. **Cierre:** cuando el arreglo llega a Demo, el Equipo del PO repite la prueba original (`/po-team:validar --revalidar #240`), comenta en el mismo ticket (que se corrigio, veredicto, fotos antes / despues) y **te pregunta si lo cierra**; nunca lo cierra solo. Sin Equipo del PO, ese comentario lo hace el dueño del producto del Dev Team. Si va a un ambiente del cliente, sale con su pase (`/dev-team:pase ...`).
8. **De vuelta al cliente:** la persona de mesa le informa a ACME, y lo aprendido se guarda con `/mesa-servicios:brain ingest BUG-2026-003`.

> **Si el error te lo cuentan a ti** (en una reunion, por correo) y nadie lo reprodujo, el Equipo del PO lo confirma primero: `/po-team:validar --reproducir "{lo que reportaron}"`. Solo con **REPRODUCIDO** se registra el bug; si viene de un cliente, puede convenir devolverlo a la Mesa.

**Recibes:** la certeza de que el error existe, la correccion con evidencia antes/despues, una prueba automatica que impide que vuelva y el caso guardado en la memoria del negocio.

---

## Caso 3 — Un incidente urgente en produccion

**Situacion:** en produccion, ningun analista de ACME puede aprobar operaciones desde esta mañana.

```
/dev-team:hotfix los analistas de ACME no pueden aprobar operaciones en produccion --ambiente prod
```

**Que hace el Dev Team:** la urgencia acorta los tiempos, **nunca los controles**.

1. Pruebas acota y reproduce con evidencia (si el incidente productivo es observable, basta la evidencia del estado actual).
2. El jefe de equipo presenta un plan corto: que se cambia, el riesgo y **como se revierte si empeora**. Espera tu OK.
3. Correccion minima, sin mejoras "de paso".
4. Pruebas revalida con par antes/despues; seguridad revisa si el cambio es sensible; el jefe de equipo integra.
5. El pase formal al ambiente sigue yendo por `/dev-team:pase`.

> Si el reporte llego primero a la Mesa, la Mesa puede reproducirlo en produccion solo con la autorizacion registrada de esa corrida, y el probador se detiene antes de cada paso que cambie algo.

---

## Caso 4 — Hay que corregir o revertir datos de un cliente

**Situacion:** "la operacion 10432 de ACME se giro por error; hay que revertirla".

1. **La Mesa arma los scripts, sin ejecutarlos:**

   ```
   /mesa-servicios:sql script SOL-2026-03-12-4
   ```

   → el de los datos **solo mira** (confirma la operacion, su estado real y lo que depende de ella), te presenta el plan y arma el paquete en el formato de la casa: ordenado por motor, base y tipo de operacion, que se puede ejecutar dos veces sin daño, con verificacion antes y despues, su reversa y un README que dice **quien debe ejecutarlo**.
2. **Si va a un ambiente formal, entra al pase del Dev Team:**

   ```
   /dev-team:pase produccion CL — reversa operacion 10432 ACME
   ```

   → el encargado de publicaciones **revisa el formato de los scripts y puede rechazarlos** (indicando archivo, linea y regla). Si esta bien, arma `Scripts.zip` y la solicitud formal.
3. **Si el pedido se repite**, la Mesa lo deja como flujo reutilizable (`/mesa-servicios:sql flujo crear factoring/revertir-operacion`): la proxima vez basta el numero de operacion.

**Lo que nunca pasa:** ni la Mesa ni nadie "arregla rapido" algo directo en la base. Siempre hay script revisado, reversa y alguien autorizado que lo ejecuta.

---

## Caso 5 — Un proyecto nuevo completo

**Situacion:** la Mesa cerro un proyecto grande para ACME (`PRY-2026-014 — Portal de proveedores`) y comercial lo aprobo. Hay que armarlo desde cero.

1. **El Dev Team crea el proyecto desde el documento de la Mesa:**

   ```
   /dev-team:new-project PRY-2026-014-Portal-Proveedores.pdf
   ```

   El arquitecto lee el documento completo, te señala vacios, te propone en una pagina en cuantas partes se divide y con que se construye, y ajusta hasta tu OK. Luego se crean los repositorios y el dueño del producto arma las historias **indicando de que parte del documento sale cada una**.
2. **Si es un plan por fases** y quieres el tablero de GitHub completo (fases, categorias, historias, hitos):

   ```
   /dev-team:project crear PRY-2026-014-Portal-Proveedores.pdf
   ```

3. **Diseño antes de programar:** `/dev-team:design portal de proveedores ...` → dos o tres caminos visuales reales, eliges uno, y recien ahi se construye exactamente eso.
4. **Construccion por sprints** (ver el Caso 14 del manual del Dev Team). Si usas el Equipo del PO, este sigue cada sprint con sus fechas de pase, valida en Demo y consigue el VB del cliente ( `/po-team:sprint planificar 1` , `/po-team:tablero` ). Al tener el producto en un ambiente demo, **Marketing** prepara su lanzamiento (Caso 6).

---

## Caso 6 — Lanzar un modulo o contarle las mejoras a la cartera

**Situacion:** el modulo Confirming 2.0 esta publicado en el ambiente demo y hay que presentarlo al CEO de una financiera y a la cartera de clientes.

```
/marketing:propuesta-comercial "Confirming 2.0 para el CEO de una financiera" --pais PE
```

**Que hace Marketing:**

1. Brief de una ronda (como mucho cinco preguntas: quien decide, que decision debe tomar, que se puede decir, precios de comercial, fecha) y **plan primero**.
2. Lee las fuentes reales: la memoria, el backlog y el diseño del Dev Team; lo que pidieron los clientes, las maquetas y las propuestas de la Mesa; y captura pantallas en el **ambiente demo con datos ficticios**.
3. **Mapa del producto** → tu apruebas. **Fichas por parte** → tu apruebas. Sin esas dos aprobaciones no se produce ninguna pieza.
4. Produce en paralelo el **video keynote** (apertura → problema → una parte por capitulo → prueba → hoja de ruta → cierre), un **clip de 6 a 15 segundos por parte**, el **deck de propuesta** (una seccion por parte) y el **one-pager**. Cada pieza pasa por tu aprobacion.

**Variantes:** `/marketing:video teaser` (pieza corta), `/marketing:pieza redes linkedin --carrusel` (una parte por lamina), `/marketing:investigar competencia confirming --pais PE` (benchmark con fuente y fecha), `/marketing:estado` (que espera tu aprobacion).

---

## Caso 7 — Una consulta tecnica puntual

**Situacion:** un desarrollador tiene una duda concreta en un repositorio que no esta bajo un proyecto del Dev Team, o quiere una segunda opinion experta.

```
/architect revisa mi patron de repositorio, creo que hay un problema con el ciclo de vida
del DbContext
/architect diseña el esquema de base de datos de un sistema de facturacion multi-empresa
en PostgreSQL
/architect voy a migrar de .NET 6 a .NET 9, ¿que cambios incompatibles debo esperar?
```

**Recibes:** hallazgos clasificados por gravedad (**Critico**, **Importante**, **Sugerencia**, **Buena practica**), cada uno con el por que y la solucion concreta; si hay varias opciones validas, dos o tres con sus pros y contras. Si se lo pides, implementa.

> **¿Arquitecto Senior o el arquitecto del Dev Team?** El del Dev Team trabaja dentro del equipo: diseña proyectos, decide en cuantas partes se divide y deja su decision para que el resto construya. El Arquitecto Senior es un **consultor de una vez**, para cualquier repositorio, sin tablero ni carpeta de trabajo.

---

# Parte 3 — Preguntas frecuentes y problemas comunes

## Preguntas frecuentes

**¿Tengo que instalar los cinco equipos?** No. Instala solo los que usas. La Mesa sirve sola; el Dev Team sirve solo; el Equipo del PO sirve solo (si hay Mesa, lee sus tickets; si no, parte de tu pedido o de un documento); Marketing rinde mas si existen las carpetas del Dev Team o de la Mesa, porque de ahi saca el producto real.

**¿Pueden convivir varios equipos en la misma computadora y el mismo proyecto?** Si. Cada uno usa su propia carpeta de trabajo (`.mesa/`, `.po-team/`, `.coordination/`, `.marketing/`) y ninguno escribe en la carpeta de otro. Marketing solo lee las de los demas, y el Equipo del PO solo lee la memoria del negocio de la Mesa.

**¿Tengo que saber que integrante hace cada cosa?** No. Usa el comando `start` del equipo y describe lo que necesitas. El equipo se organiza.

**¿Algun equipo le escribe al cliente o envia correos por su cuenta?** Nunca. Te deja las preguntas, la propuesta o el correo listos; tu los envias.

**¿Funcionan con Azure DevOps ademas de GitHub?** La Mesa, el Equipo del PO y el Dev Team si; se elige al configurar. El Dev Team tambien puede trabajar "solo local". Marketing no usa tablero.

**¿Puedo usar la Mesa sin instalar nada?** Hay una version simplificada para Claude web (claude.ai) que se habilita para toda la organizacion (ver seccion 7). No tiene grabacion de pruebas ni conexion al tablero; para eso esta el plugin.

**¿Puedo pedirle algo a un integrante a traves de la sesion?** Si. "Dile al probador que reproduzca la SOL-2026-03-11-2" funciona tal cual. Lo unico que no se pasa asi es la aprobacion de un permiso de Claude Code en pantalla.

**¿Por que en pruebas el equipo aprueba o gira sin preguntarme?** Porque en un ambiente de pruebas son acciones de prueba normales. En productivo es al reves: cada accion delicada espera tu autorizacion.

**Deje una pregunta sin responder. ¿Se pierde?** No. Aparece en la lista de pendientes al abrir la siguiente sesion.

**¿Por que no veo el trabajo de los equipos en los repositorios?** Porque no va ahi. Al repositorio solo va codigo, migraciones de base de datos y documentacion tecnica. Asi nada interno se publica por error.

**¿Cuanto cuesta usarlos?** Depende del plan de Claude de tu cuenta. Los equipos ya vienen en el "cerebro" de tamaño justo. Lo que mas ahorra: sesiones cortas por tarea, mantener al dia la memoria de cada equipo y hacer las preguntas simples directo, sin el comando `start`.

**¿Como me entero de que hay una version nueva?** El Dev Team y el Equipo del PO te avisan al abrir una sesion. Para todos, la actualizacion es la del Paso 8.

## Si algo no anda

- **"Escribo el comando y no aparece"** → reinicia Claude Code (`/exit` y `claude`) y revisa con `claude plugin list` que el equipo este instalado.
- **"Plugin not found" al actualizar** → usa el nombre completo: `claude plugin update dev-team@faast-marketplace`.
- **"Actualice pero el equipo sigue igual"** → cierra y abre la sesion: todo se carga al inicio.
- **"Error de acceso al agregar el marketplace"** → tu cuenta de GitHub necesita acceso a la organizacion `faast-app`.
- **"Un integrante dice que no puede guardar en esa carpeta"** → es el control que impide guardar el trabajo del equipo dentro de un repositorio. Acepta la propuesta de moverla (ver Anexo B).
- **"Claude Code me pide aprobar un permiso"** → es normal y lo apruebas tu en pantalla.
- **"El probador no puede abrir el navegador"** → reinicia la sesion; si persiste, pide `/dev-team:setup playwright`, `/mesa-servicios:setup playwright` o `/po-team:setup playwright`.
- **"El revisor del PO no puede entrar al sistema"** → completa la cuenta en su archivo de accesos y reconecta el navegador (`/mcp`) o reinicia la sesion (ver el manual del Equipo del PO).
- **"El video se ve chico o borroso"** → pide que lo regraben; no cuenta como evidencia.
- **"El equipo no encuentra mi proyecto"** → abriste Claude Code en otra carpeta. Abrelo en la carpeta de trabajo (ver Paso 3) o registra la carpeta (ver Anexo C).

Para lo especifico de cada equipo, ver la seccion "Si algo no anda" de su manual.

---

# Anexo tecnico (para quien instala y configura)

## A. Instalacion y actualizacion por linea de comandos

```
# 1. Agregar el marketplace (una sola vez por computadora)
claude plugin marketplace add faast-app/faast-claude-marketplace

# 2. Instalar los plugins
claude plugin install mesa-servicios@faast-marketplace
claude plugin install po-team@faast-marketplace
claude plugin install dev-team@faast-marketplace
claude plugin install marketing@faast-marketplace
claude plugin install senior-backend-architect@faast-marketplace

# 3. Verificar
claude plugin list

# 4. Actualizar cuando haya cambios publicados en main
claude plugin marketplace update faast-marketplace
claude plugin update <plugin>@faast-marketplace
# → reiniciar la sesion de Claude Code (agentes, hooks y MCP cargan al inicio)
```

Dentro de una sesion, las formas equivalentes son `/plugin marketplace add ...`, `/plugin install ...` y el menu `/plugin`.

**Requisitos base:** Claude Code con sesion iniciada, `git`, `python3` (los hooks de los equipos estan escritos en Python) y Node.js. Cada equipo valida e instala el resto con su agente `setup` y una confirmacion: Dev Team (Docker, `gh` / `az`, clientes de BD, Playwright, `pipx`), Mesa (`gh` / `az`, `ffmpeg` + Whisper, `pandoc`, LibreOffice, clientes de BD de solo lectura, Playwright), Equipo del PO (`gh` con permisos `project` y `read:project` o `az` con Boards, LibreOffice, Playwright, `ffmpeg` opcional), Marketing (Node ≥ 22, FFmpeg, HyperFrames, LibreOffice, Playwright). El servidor MCP de Playwright viene incluido en los plugins dev-team, mesa-servicios, po-team y marketing; si tenias uno `playwright` registrado a mano, quitalo con `claude mcp remove playwright` para no duplicar herramientas.

## B. Las carpetas de cada equipo

Cada equipo guarda su trabajo en una carpeta oculta dentro de la **carpeta del proyecto** (o de la mesa), **nunca dentro de un repositorio git**. Antes de crearla se verifica con `git -C "<carpeta>" rev-parse --is-inside-work-tree`, que debe fallar.

```
~/Documents/Trabajo/Faast/Proyectos/Backoffice/     ← carpeta del proyecto (NO es un repo)
├── .coordination/          Dev Team
├── .po-team/               Equipo del Product Owner (tambien puede ir en una carpeta propia del PO)
├── .marketing/             Marketing
├── micro-backoffice/       repo git (solo codigo, migraciones, documentacion tecnica)
└── front-backoffice/       repo git

~/Mesa/mesa-faast/                                  ← carpeta de la mesa (NO es un repo)
└── .mesa/                  Mesa de Servicios
```

| Carpeta | Equipo | Lo principal que contiene |
|---|---|---|
| `.coordination/` | Dev Team | `config.json` (fuente de verdad: topologia, tablero, ambientes, pase, deploy), `handoffs/`, `wiki/`, `evidence/`, `design/`, `pases/`, `flows/`, `docs-negocio/`, `retros/`, `metrics/`, `backlog.md` |
| `.mesa/` | Mesa de Servicios | `config.json`, `correlativos.json`, `backlog.md`, `solicitudes/{ID}/`, `flujos/`, `flujos-sql/`, `evidencia/`, `handoffs/`, `metrics/` |
| `.po-team/` | Equipo del Product Owner | `config.json` (tablero, ciclo de pase, ambientes con su clase, clientes, capacidad), `accesos.env`, `backlog.md`, `sprints/`, `retros/`, `tickets/{n}/`, `validaciones/`, `tablero/` (paneles, `snapshots/`, `informes/`), `documentos/`, `manuales/`, `project-state/`, `handoffs/`, `metrics/` |
| `.marketing/` | Marketing | `config.json`, `correlativos.json`, `campanas/{ID}/`, `tendencias/`, `marca/`, `handoffs/`, `metrics/` |

**Secretos que nunca entran a git:** `.coordination/dba-access.json`, `.coordination/qa-secrets.env`, `.mesa/accesos.env`, `.mesa/db-access.json`, `.po-team/accesos.env`, los `setup-status.json` y las carpetas de evidencia. Como las carpetas viven fuera de los repos, ninguno llega a un repo.

**Protecciones automaticas:** un hook de cada equipo bloquea escribir en su carpeta si esta dentro de un repo, y al iniciar sesion cada plugin agrega localmente `.coordination/`, `.mesa/`, `.marketing/` y `.playwright-mcp/` (entre otros) a `.git/info/exclude` del repo abierto, para que nada se commitee por error. El Equipo del PO hace lo mismo con `.po-team/`. Si una carpeta antigua quedo dentro de un repo, la sesion propone moverla con `mv` y registrarla (Anexo C); sacarla del historial del repo es un PR aparte, solo con OK.

**Evidencia en el tablero:** en GitHub se publica en la rama huerfana `evidence` del repositorio configurado (permanente, jamas mergeada) y se embebe en el issue; en Azure DevOps, como adjunto embebido en el item. Nunca en una rama de codigo.

## C. Registro personal de carpetas

Cuando los repositorios no viven dentro de la carpeta del proyecto (por ejemplo, repos en `~/Repositorios/` y proyectos en `~/Proyectos/`), cada plugin necesita saber donde esta su carpeta de trabajo. Eso se registra **por usuario, sin tocar el repo**, en un archivo personal por equipo:

| Equipo | Archivo personal | Valor de cada entrada |
|---|---|---|
| Dev Team | `~/.claude/dev-team.config.json` | La carpeta que contiene `.coordination/` |
| Mesa de Servicios | `~/.claude/mesa-servicios.config.json` | La carpeta que contiene `.mesa/` |
| Equipo del Product Owner | `~/.claude/po-team.config.json` | La carpeta que contiene `.po-team/` |
| Marketing | `~/.claude/marketing.config.json` | La carpeta que contiene `.marketing/` |

```
{
  "team": { "models": { "architect": "opus" } },
  "proyectos": {
    "~/Repositorios/front-backoffice": "~/Proyectos/backoffice",
    "~/Repositorios/micro-factoring":  "~/Proyectos/backoffice"
  }
}
```

La clave es la ruta de un repo o carpeta; el valor, la carpeta del proyecto. **Gana el prefijo mas largo.** El orden de busqueda es: (1) este registro → `proyectos`; (2) el puntero legado (`.coordination-root` / `.mesa-root` / `.po-team-root`); (3) subir directorios buscando la carpeta con `config.json`, prefiriendo la que esta fuera de repos. `/dev-team:onboard` y `/marketing:start` completan el registro cuando corresponde, y la sesion lo propone al detectar una carpeta dentro de un repo.

## D. Modelos de IA por integrante

| Equipo | Por defecto | Se puede cambiar |
|---|---|---|
| Dev Team | `haiku` para setup y tech-writer (fijos); `sonnet` para el resto | Si, salvo setup y tech-writer. Lo habitual: `architect` a `opus` en proyectos complejos |
| Mesa de Servicios | `sonnet` para los 10 | Si, por integrante (p. ej. `analista` a `opus`) |
| Equipo del Product Owner | `haiku` para setup (fijo); `sonnet` para el resto | Si, salvo setup (p. ej. `refinador` a `opus`) |
| Marketing | `sonnet` para los 8 | Si, por integrante |

| Equipo | Por defecto | Se puede cambiar |
|---|---|---|
| Arquitecto Senior Backend | `sonnet` | Definido en el agente del plugin |

El cambio se hace con `team.models.{integrante}` en la configuracion del proyecto (`.coordination/config.json`, `.mesa/config.json`, `.po-team/config.json` o `.marketing/config.json`) o en el archivo personal del Anexo C; **si hay de los dos, gana el del proyecto**. Valores: `haiku` (economico), `sonnet` (intermedio), `opus` (potente). Requiere una sesion nueva para aplicarse.

## E. Problemas tecnicos comunes

| Sintoma | Causa probable | Que hacer |
|---|---|---|
| `Failed to update plugin: Plugin ... not found` | Falta el nombre del marketplace | `claude plugin update <plugin>@faast-marketplace` |
| Actualice pero los agentes siguen igual | Agentes, hooks y MCP cargan al inicio de sesion | Reiniciar; verificar con `claude plugin list` |
| No aparecen las herramientas `browser_*` o hay dos `playwright` | Sesion vieja o MCP personal duplicado | Reiniciar; `claude mcp list` debe mostrar uno por plugin; quitar el personal con `claude mcp remove playwright` |
| Un hook bloquea escribir en `.coordination/`, `.mesa/`, `.po-team/` o `.marketing/` | Esa carpeta esta dentro de un repo git | Moverla con `mv` fuera del repo, registrarla en `~/.claude/<plugin>.config.json` → `proyectos` y reiniciar |
| El equipo usa una carpeta equivocada | Hay una carpeta huerfana (sin `config.json`) mas cerca que la canonica | Registrar el repo en `proyectos` (Anexo C); la sesion propone fusionar la huerfana con OK |
| `gh` no tiene acceso al Project | Falta el permiso `project` | `gh auth refresh -s project,read:project` |
| Los hooks no registran actividad | Falta `python3` | Instalar Python 3 y reiniciar |
| Un integrante no uso el modelo configurado | Clave mal escrita o sesion vieja | Verificar `team.models.{integrante}` exacto y abrir una sesion nueva |

| Sintoma | Causa probable | Que hacer |
|---|---|---|
| El video de evidencia sale chico en una esquina | La traza se encendio antes que el video | Seguir "Video a tamaño real" en la skill `visual-evidence` (Dev Team), `evidencia-reproduccion` (Mesa) o `evidencia-po` (Equipo del PO) y regrabar |

Para el detalle tecnico de cada equipo (configuracion completa, estructura de carpetas, consumo), ver el Anexo tecnico de su manual: Dev Team, Mesa de Servicios, Equipo del Product Owner, Marketing.